Low-latency order book: results and analysis

Abhiroop | 24BCS10287 | abhiroop.24bcs10287@sst.scaler.com | 17 October 2026

Setup and correctness

A uses ordered maps for orders and levels. B uses sorted level vectors, reusable order slots and an unordered ID index. C uses fixed price-indexed quantity arrays, a custom flat order table and cached BBO. All use integer ticks, C++17 and -O2.

Environment: Ubuntu 24.04 ARM64 Linux VM, GCC 13.3.0, LinuxKit 7.0.12 and perf 6.8.12. No CPU pinning. The full million-operation verification passed on Linux: complete levels and BBO agree after every operation. Edge tests cover aggregation, cancellation, price boundaries, sparse levels and hash-table collisions and growth.

Workload: seed 42; 700,818 adds, 199,505 live-order cancels, 99,677 BBO queries, plus 10,000 depth-query pairs. Prices are uniform within the PDF's bands; peak live orders are 501,313. All runs finish at bid 99.99 x 2747, ask 100.01 x 2018, checksum 10960379378034558822.

Results

Metric	A: maps	B: vectors	C: arrays
Total time (ms)	223.142	77.424	51.404
ns / add	187.534	72.069	126.359
ns / cancel	641.452	193.972	75.300
ns / BBO	17.779	18.888	18.756
IPC	N/A	N/A	N/A
Branch misses (%)	N/A	N/A	N/A
L1-dcache-load-misses	N/A	N/A	N/A
Cache misses	N/A	N/A	N/A
Page faults	16386	14795	15790
Task-clock (ms)	183.09	113.14	81.07

Chrono values are medians of five trials. Page faults and task-clock are perf stat -r 5 averages, covering the whole process. Hardware cycles, instructions, branches, branch misses, cache misses, L1 misses and dTLB misses are not supported in this VM; N/A is not zero.

Which is fastest?

C has the lowest total time and cancellation latency; B has the lowest measured add latency. B's order slots append/reuse contiguous storage, while C probes a much larger flat table. C avoids a level search during cancellation. These are explanations consistent with the design, not isolated causal measurements. A has the lowest BBO average, but all three values are close to clock overhead; this is not convincing evidence that its BBO implementation is better.

Operation timings use a separate replay with two clock reads per call; total time has no such reads. Do not sum the operation averages to reconstruct total time. Empty intervals average 14-15 ns, their median is zero, and the smallest positive observed interval is 41 ns. Quantization and instrumentation matter here. A's total trials range from 155 to 254 ms; allocator state and VM scheduling limit repeatability.

What the profiles show

The required perf record -g commands succeeded using the software event task-clock:ppp. Five total-only replays per recording provide call graphs. Kernel symbol resolution is restricted, so unresolved kernel addresses are not assigned explanations. The saved reports show where time was sampled, including setup and destruction; they are not hardware-counter measurements.

For A, add_order accounts for about 45% of inclusive samples and cancel_order about 43%. Tree insertion/rebalancing, lookup and erasure appear in those paths. For B, lower_bound accounts for about 66% of self samples, making price search a clear target. For C, about 71% of self samples fall in replay, which contains inlined book work; about 9% of inclusive samples are under initial table allocation. Inlining prevents a precise add-versus-query split from these symbols alone.

Branch prediction and cache behavior

The available counters cannot identify which implementation has the most branch or cache misses. A follows tree pointers and makes key comparisons. B's binary search branches on randomly selected prices; its prominence in the profile suggests investigating search, but does not prove misprediction. C branches while probing live/deleted slots and scanning empty price levels.

B and C keep price data contiguous. A allocates separate tree nodes, and B still has node allocations in its unordered ID index. C avoids those nodes but touches a large preallocated order table. These layouts suggest different locality costs; they do not establish a measured cache-miss ranking. Uniform prices and net order growth fill the ladders, so B often changes existing quantities rather than shifting levels, and C's empty-level scans are uncommon. This workload does not represent all cancellation-heavy or sparse books.

Page faults and memory

A has the most measured page faults (16,386), followed by C (15,790) and B (14,795). Whole-process counts include replay generation and first touches, not just order operations. A allocates many tree nodes; C initializes its large slot array before chrono timing. The results are compatible with allocation/first-touch costs, but do not show that every allocation caused a fault, that faults required disk I/O, or that A has the largest resident memory. No RSS measurement was taken.

Choice and next experiments

For this fixed-price-band problem, C is the starting choice: lowest total and cancel latency, constant-time cached BBO, compact price storage, and direct updates. The tradeoffs are the flat table's reserved memory, hash collision/rehash tails, O(P) replacement scans, and more code to maintain. B is simpler, adds faster in the instrumented run, and deserves consideration if insertion dominates. An actual trading-system choice needs tail-latency and capacity evidence, not just these averages.

Next, measure on native Linux with hardware counters; test churn-heavy and sparse-price workloads and latency percentiles; then try a bitmap for C's next-level search or a different search strategy for B. A pool allocator would help isolate allocation cost in A. Change one factor at a time and retain the required workload as the baseline.

Evidence: results/benchmarks.txt, results/perf-stat.txt, results/perf-report.txt and results/correctness.txt. README documents data structures, all five operation complexities, commands and timing boundaries.